Sistema observable integrado en AWS: AIOps, red y seguridad, caos y madurez

Integrantes	Santiago López Amaya · Jeisson Alejandro Fúquene Buitrago
Repositorio	github.com/json-fuquene/pipeline-otel-e2e · rama `feature/actividad-3.3` · etiqueta `v1.0`
Entorno	Sandbox AWS propio, región us-east-1, 17-10-2026 (desplegado con Terraform y destruido al finalizar)
Base	Laboratorio de la actividad 2.2 (orders y catalog instrumentados con OpenTelemetry)

Resumen ejecutivo

Se extendió el laboratorio de la actividad 2.2 hasta un sistema observable completo en AWS. Ahora tiene tres microservicios (orders, catalog y el nuevo **data-service**) sobre ECS Fargate y Amazon RDS PostgreSQL. Los tres están instrumentados con OpenTelemetry (trazas, métricas y logs correlacionados por `trace_id`) y usan las convenciones semánticas estables de base de datos. ECS Service Connect actúa como malla L7. Sobre esa base se construyó una capa AIOps (bandas de anomalía 2σ, regla de correlación, Lambda que enriquece la alerta con `trace_id` y DevOps Guru) y una capa de red y seguridad (VPC Flow Logs, Security Hub, Inspector, Golden Signals de Seguridad). La resiliencia se validó con dos experimentos de caos controlados.

3 / 3
servicios instrumentados con trazas, métricas y logs

62 s
MTTD exp. 1 (latencia en catalog, alarma SLO p99)

70 s
MTTD exp. 2 (10 % de errores en data-service)

2,6 / 5
madurez promedio autoevaluada (8 dominios)

Resultado por módulo

Módulo	Resultado	Estado
A · Arquitectura observable	Tres servicios en ECS Fargate, RDS PostgreSQL 17 con TLS obligatorio, spans DB con semconv estable, malla L7 con ECS Service Connect. GCP solo como IaC de referencia.	**Cumplido en AWS**
B · AIOps	SLIs desde logs, bandas 2σ, alarma compuesta *error_rate > baseline+2σ Y p99 > SLO*, Lambda con `trace_id` y runbook, DevOps Guru sobre data-service.	**Desplegado; la regla correlacionada no se disparó**
C · Red y seguridad	VPC Flow Logs con N-S/E-W y rechazos, alarma de tráfico rechazado (disparada por escaneos reales de Internet), Security Hub FSBP, Inspector ECR, dashboard de Golden Signals.	**Cumplido en AWS**
D · Caos y MTTD	Dos experimentos ejecutados con reversión automática; detección en 62 s y 70 s; análisis de SLO y error budget.	**MTTD ≤ 2 min**
E · Madurez	Autoevaluación en 8 dominios (escala 1–5) y roadmap a 3 meses con indicadores verificables.	**Cumplido**

1. Arquitectura observable completa (Módulo A)

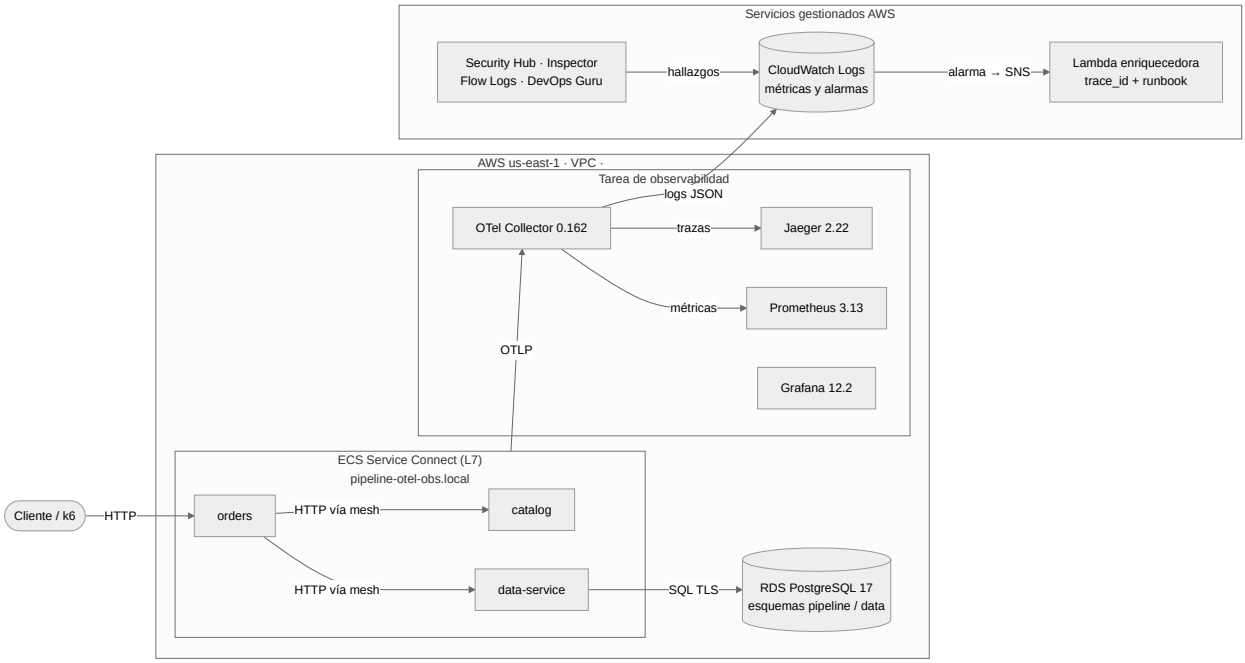


Figura 1. Arquitectura desplegada. orders y catalog también escriben en RDS (esquema `pipeline`); Grafana consulta Prometheus, Jaeger y CloudWatch.

Decisiones de diseño

Decisión	Justificación
ECS Service Connect en lugar de AWS App Mesh	App Mesh llega a fin de soporte el 30-09-2026. Service Connect usa un proxy Envoy gestionado por servicio, que da descubrimiento, reintentos y métricas L7 (solicitudes, 5xx, latencia por destino) sin cambiar el código.
data-service con esquema propio `data`	Registra cada pedido (`order_records`) y expone reportes. Un contenedor `db-bootstrap` idempotente crea el rol con mínimo privilegio. En PostgreSQL 16+ esto requiere `GRANT … WITH INHERIT FALSE, SET TRUE`.
Semconv de base de datos estable	`OTEL_SEMCONV_STABILITY_OPT_IN=http,database` produce `db.system.name`, `db.namespace`, `db.collection.name`, `db.query.summary` y el histograma `db.client.operation.duration`.
Collector como punto único	Un pipeline con `memory_limiter`, `batch` y `resource`. El conector `servicegraph` deriva el grafo de dependencias y el exportador `awscloudwatchlogs` envía logs JSON a CloudWatch.
Toda la infraestructura como código	Terraform con 6 módulos (vpc-observable, ecr, rds, ecs-observable, aiops, security-observability). Hay una referencia GCP equivalente (Cloud SQL, Cloud Monitoring, SCC) en `infra/gcp-observable`, que no se aplicó por alcance.



Figura 2. `terraform init/validate` con los 6 módulos.

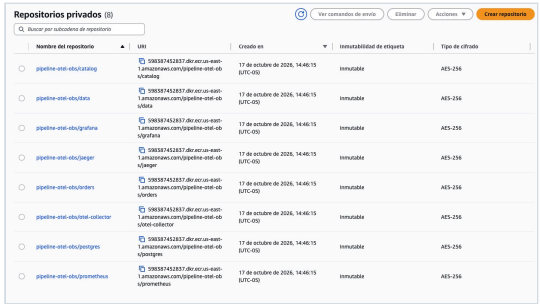


Figura 3. Ocho repositorios ECR inmutables, con escaneo al subir la imagen.

2. Evidencia de despliegue en AWS

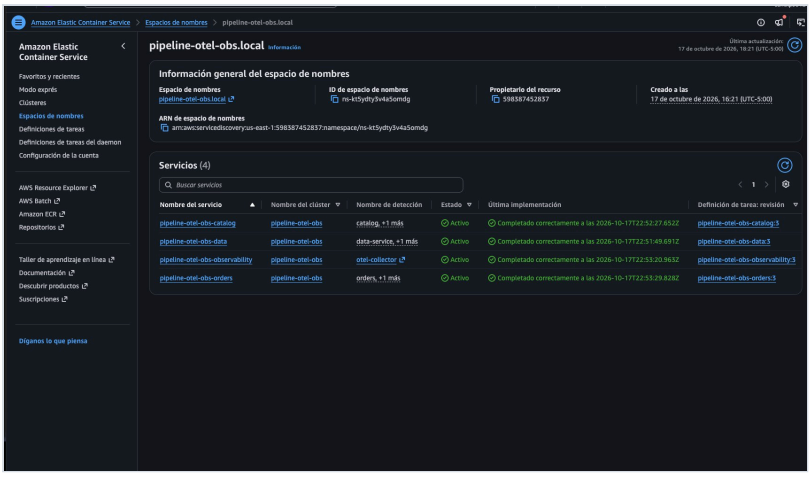


Figura 4. Espacio de nombres `pipeline-otel-obs.local` de ECS Service Connect: los cuatro servicios (catalog, data, observability, orders) activos con la revisión 3 de la tarea desplegada.

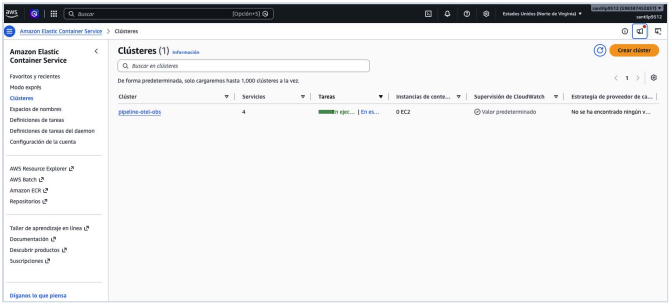


Figura 5. Clúster ECS `pipeline-otel-obs` con 4 servicios en Fargate (0 instancias EC2).

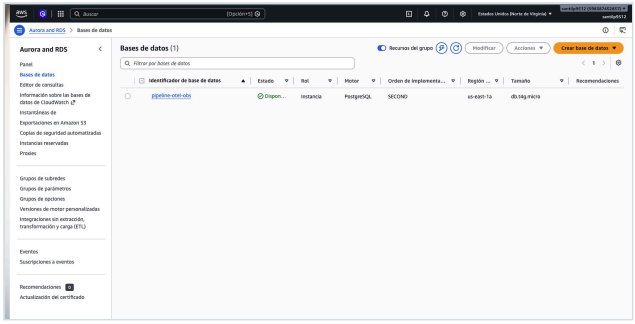


Figura 6. Instancia RDS PostgreSQL `pipeline-otel-obs` (db.t4g.micro) disponible.

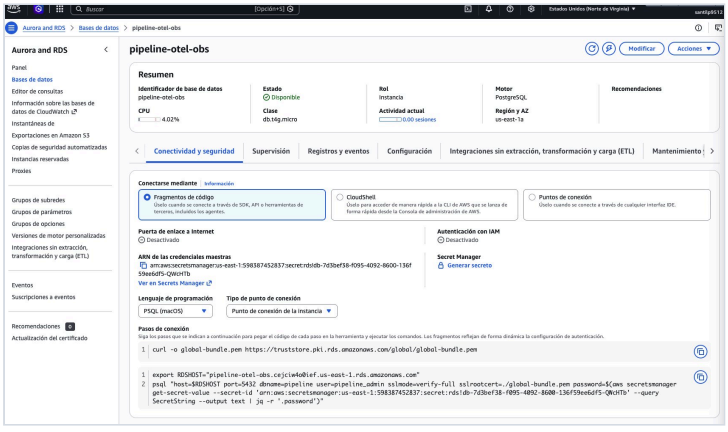


Figura 7. RDS sin acceso público y con credencial maestra gestionada en Secrets Manager. Los servicios se conectan con `sslmode=verify-full`.



Figura 8. Endpoints resueltos y *health checks* `ready` (orders verifica database y catalog).

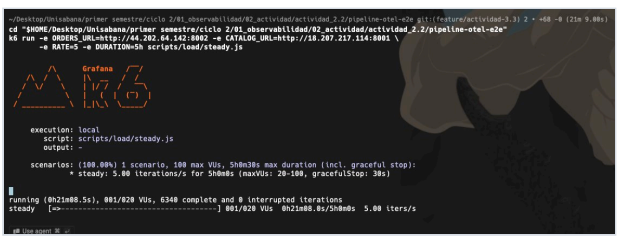


Figura 9. Carga estable con k6 (5 iteraciones/s) para la línea base de AIOps.

3. Tres pilares correlacionados: trazas y spans de base de datos

Cada `POST /orders` genera una traza distribuida de 26 spans: orders → catalog (consulta de producto) → orders (INSERT del pedido) → data-service (registro analítico). Cada salto queda instrumentado (HTTP servidor y cliente, SQLAlchemy y psycopg) con spans de negocio (`orders.order.create` , `data.record.create`).

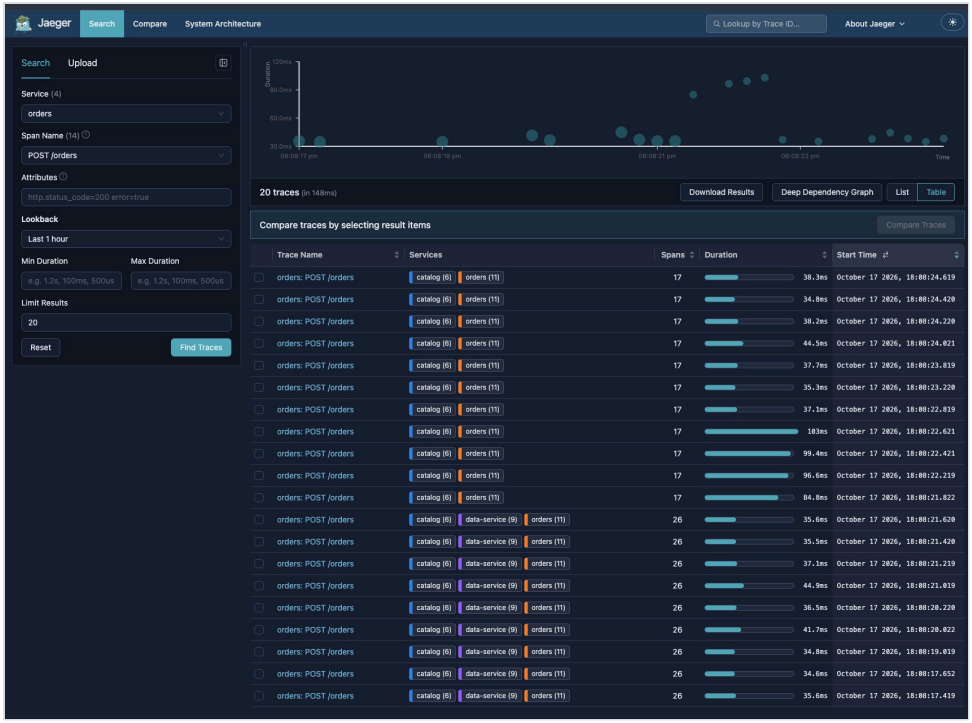


Figura 10. Jaeger: trazas `orders: POST /orders` con 3 servicios (catalog, data-service y orders) y duración típica de 35–45 ms.

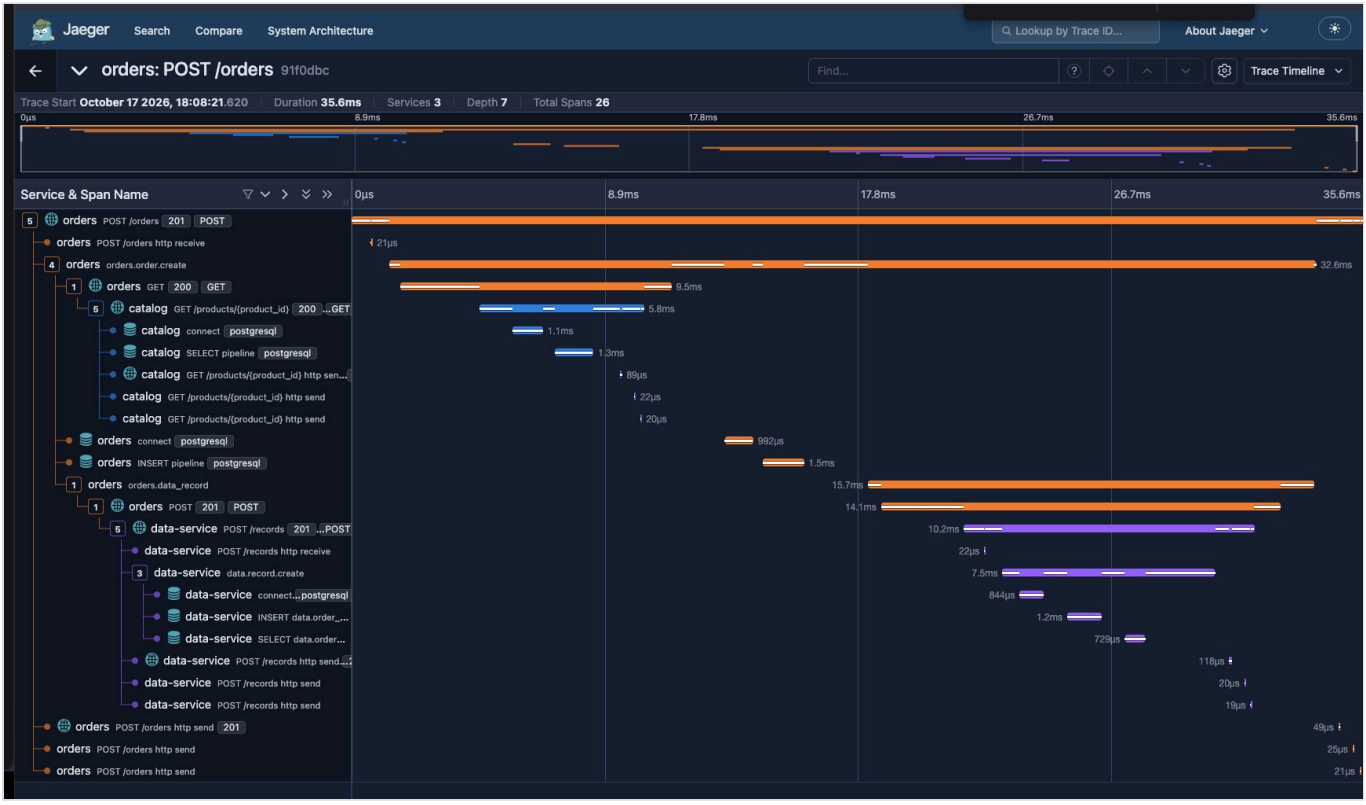


Figura 11. Una traza completa (35,6 ms, profundidad 7). Los spans de base de datos de cada servicio cuelgan de su span HTTP, de modo que la latencia se puede atribuir a red, aplicación o base de datos.

3. Tres pilares (continuación): DB semconv, logs y métricas

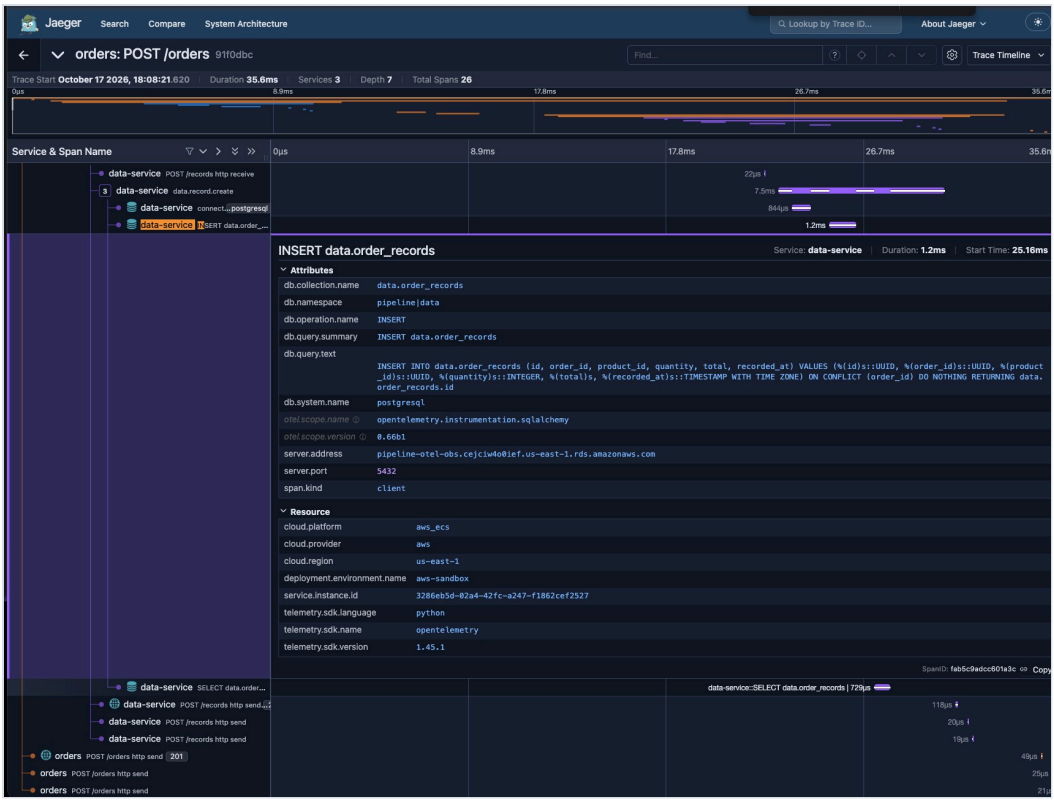


Figura 12. Span `INSERT data.order_records` con los atributos de la convención estable: `db.system.name=postgresql`, `db.namespace=pipeline|data`, `db.collection.name`, `db.query.summary`, `db.query.text` parametrizado, `server.address` del endpoint RDS y recursos `cloud.platform=aws_ecs`.

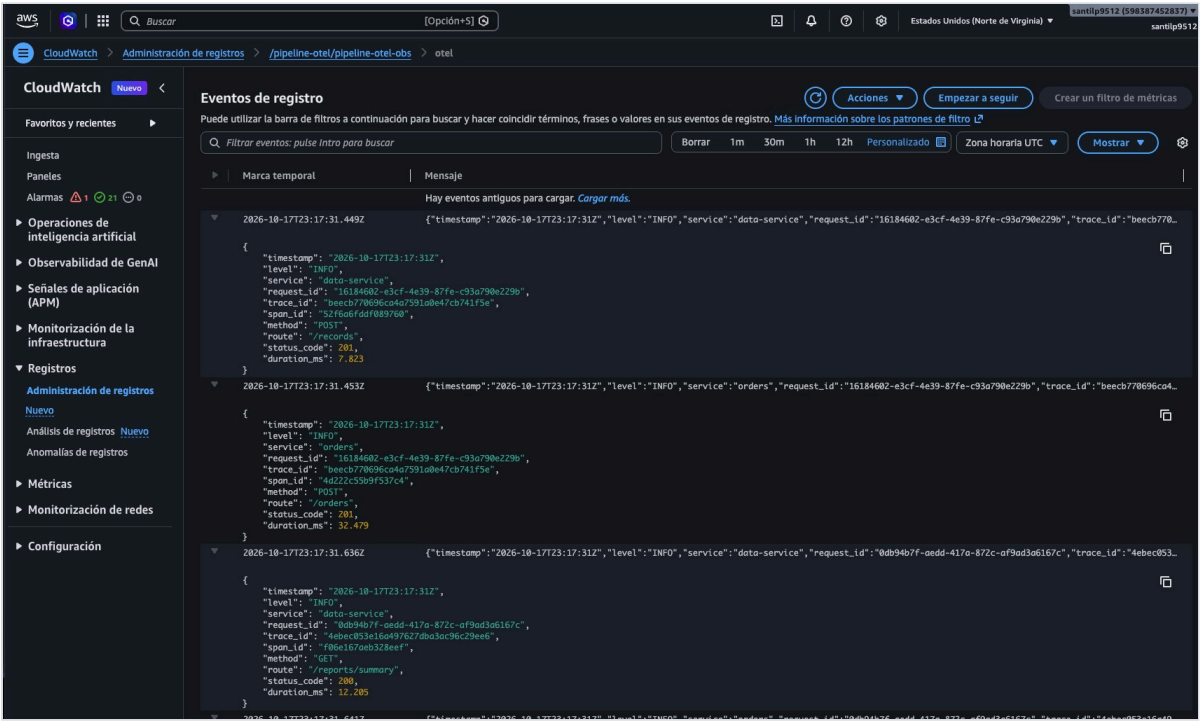


Figura 13. CloudWatch Logs: logs JSON estructurados con `trace_id` y `span_id`. El mismo `trace_id` aparece en orders y en data-service, lo que permite pivotar de un log a la traza en Jaeger. Estos logs alimentan los *metric filters* de AIOps.

4. Visualización: servicios, SLO y grafo de dependencias

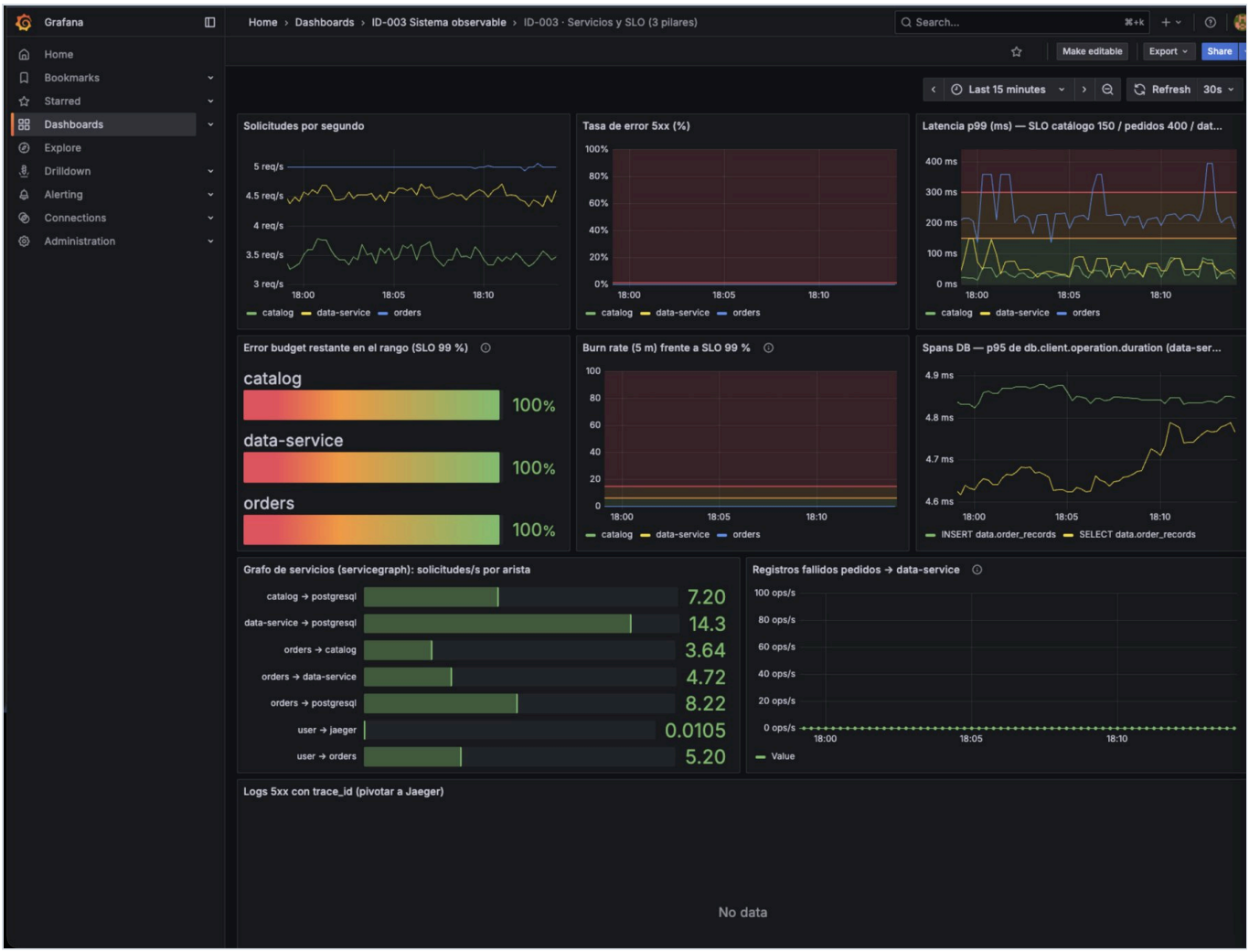
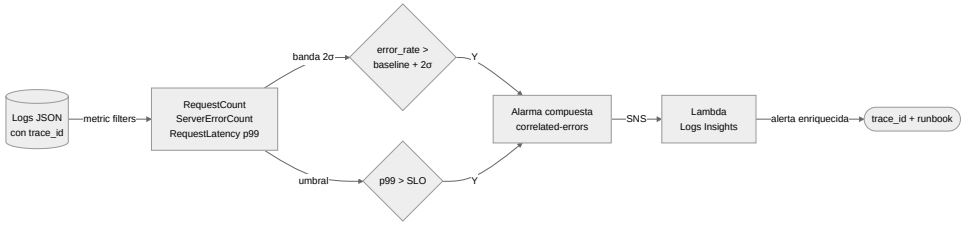


Figura 14. Dashboard **ID-003 · Servicios y SLO**, generado por código y versionado. Muestra tráfico por servicio, tasa 5xx, p99 frente al SLO de cada servicio (catalog 150 ms, orders 400 ms, data-service 300 ms), error budget restante, burn rate a 5 min, p95 de `db.client.operation.duration` por operación, grafo de servicios derivado de las trazas (incluye las aristas hacia postgresql) y logs 5xx con `trace_id`.

SLI / SLO	Definición (reglas de Prometheus)
Disponibilidad 99 %	`sli:errors:rate1m` = 5xx / total por servicio. El error budget es el 1 % de las solicitudes de la ventana y el burn rate es `error_ratio / 0,01`.
Latencia p99	`histogram_quantile(0.99, http_server_request_duration)` comparado con el SLO de cada servicio.

5. AIOps: detección de anomalías y correlación (Módulo B)



Cada servicio tiene **tres capas de alarmas** en CloudWatch. Hay alarmas estáticas (tasa de error > 1 % y p99 > SLO) y alarmas de anomalía con `ANOMALY_DETECTION_BAND(x, 2)`, es decir, una banda de ±2σ aprendida por CloudWatch. Por encima están dos alarmas compuestas: `correlated-errors` (anomalía de error **Y** p99 > SLO, la regla pedida) y `latency-degradation` (p99 > SLO **Y** anomalía de latencia). Las compuestas publican en SNS. Una Lambda consulta Logs Insights, extrae el `trace_id` de una solicitud fallida en la ventana y publica una alerta enriquecida con evidencia y runbook. Además, Amazon DevOps Guru analiza el servicio data-service por etiqueta (`DevOps-Guru-pipeline-otel-obs=data-service`).

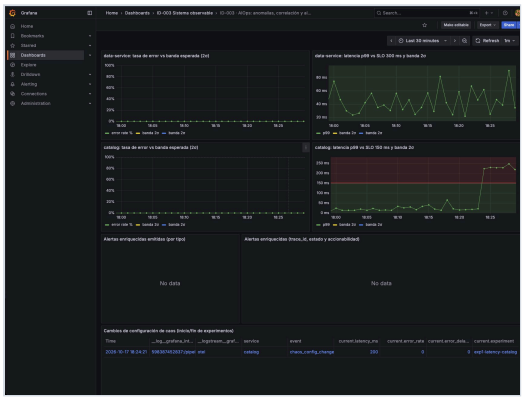


Figura 15. Dashboard AIOps durante el experimento 1. El p99 de catalog sube de ~20 ms a ~230 ms y cruza el SLO de 150 ms. La tabla inferior registra el evento `chaos_config_changed`.

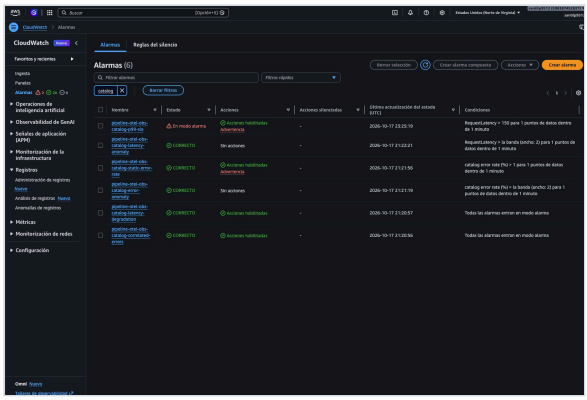


Figura 16. Alarmas de catalog. `catalog-p99-slo` pasa a ALARM, mientras que las alarmas de anomalía y las compuestas permanecen en OK.

Ruido: umbrales estáticos frente a regla correlacionada

Episodio	Notificaciones con umbrales estáticos	Regla correlacionada
Exp. 1: latencia en catalog	2 (catalog-p99 y orders-p99, síntoma en cascada)	0
Exp. 2: errores en data-service (corrida 1)	2 (static-error-rate y p99-slo)	0
Exp. 2: errores en data-service (corrida 2)	2 (static-error-rate y p99-slo)	0
Picos aislados de p99 de orders en carga estable (Fig. 14)	Picos que superan el SLO de 400 ms: candidatos a falsas alarmas estáticas	0

Análisis honesto. La regla correlacionada eliminó el 100 % del ruido, incluida la alarma en cascada de orders y los picos aislados de p99. Pero también dejó de detectar los dos incidentes reales, así que no es una reducción de ruido válida. La causa está en la entrada de anomalía. El *metric filter* `ServerErrorCount` no publica un valor por defecto, de modo que en estado estable la métrica no tiene datos. El modelo se entrenó casi sin puntos y la banda 2σ quedó vacía (las series «banda 2σ» de la Fig. 15 no se dibujan), por lo que la alarma de anomalía nunca pudo transicionar. Hay dos correcciones identificadas: `default_value = 0` en el filtro y un entrenamiento de al menos 24 h antes de evaluar. Hasta entonces, la alarma estática de tasa de error (MTTD de 70 s) cumple la función de detección.

6. Observabilidad de red y seguridad (Módulo C)

Capacidad	Implementación	Señal / alerta
VPC Flow Logs	Formato de 17 campos hacia CloudWatch Logs. Los *metric filters* separan el tráfico este-oeste (origen y destino en 10.63.0.0/16) del norte-sur, y los flujos `REJECT`.	`RejectedFlows`, `EastWestBytes`, `AcceptedBytes`
Tráfico anómalo entre servicios	Banda de anomalía 2σ sobre `EastWestBytes` (detector entrenado) y umbral sobre flujos rechazados.	`network-east-west-anomaly`, `network-rejected-flows`
Malla L7	Métricas de ECS Service Connect por servicio y destino: solicitudes, 5xx y latencia del proxy Envoy.	Dashboard **ID-003 · Red L7**
AWS Security Hub	Estándar *AWS Foundational Security Best Practices*. EventBridge lleva los hallazgos a un grupo de logs propio.	`security-high-findings`
Amazon Inspector	Escaneo de vulnerabilidades de las imágenes en ECR. Una Lambda publica la métrica `ActiveCVEs` por severidad.	CVEs activos
Autenticación fallida	*Metric filter* sobre los logs de Grafana (`invalid username or password`).	`security-failed-auth`

Golden Signals de Seguridad

El dashboard **ID-003 · Golden Signals de Seguridad** reúne las tres señales que pide el enunciado. La primera son los intentos de autenticación fallidos, a partir de la métrica del *filter* de Grafana. La segunda es el tráfico N-S frente a E-W (bytes aceptados totales frente a intra-VPC, con N-S calculado como diferencia) junto con los flujos rechazados. La tercera son los CVEs activos por severidad según Inspector, con los hallazgos de Security Hub de severidad alta o crítica. El script `scripts/security/generate_security_signals.py` genera señales controladas, como logins fallidos y sondeo de puertos cerrados, para validar las alertas.

Hallazgo real durante el laboratorio

Durante la sesión, la alarma `network-rejected-flows` se disparó sin ninguna inyección. Los Flow Logs mostraron flujos `REJECT` desde direcciones públicas de Internet contra las IP públicas de las tareas, es decir, escaneo automatizado de puertos. Los *security groups* solo admiten la IP del equipo (`allowed_cidr /32`), por lo que el tráfico fue bloqueado y quedó visible y alertado. Es una evidencia de que la observabilidad de red detecta tráfico norte-sur anómalo en condiciones reales.

> **Alcance.** La implementación es completa en AWS. Para GCP existe la IaC equivalente (VPC Flow Logs, Security Command Center y Cloud Monitoring) en `infra/gcp-observable`, pero no se aplicó.

Limitaciones de seguridad identificadas

- No se integró CloudTrail, por lo que no hay métricas de `AccessDenied` ni de `ConsoleLogin` fallido en AWS (está en el roadmap, mes 1).
- No hay correlación automática de un hallazgo de Inspector con el servicio afectado en las alertas.
- Las tareas tienen IP pública para la demostración. En producción irían detrás de un ALB con subredes privadas.

7. Chaos Engineering controlado y MTTD (Módulo D)

Mecanismo. Cada servicio tiene un *middleware* de caos que lee el parámetro SSM `/pipeline-otel-obs/chaos/<servicio>` cada 5 s, sin redespliegue. El *runner* `scripts/chaos/run_experiment.py` escribe la falla y mide el MTTD como `StateUpdatedTimestamp` de la primera alarma en ALARM menos el instante de inyección. Después calcula el SLO y el error budget en Prometheus y **revierte siempre la falla** (bloque `finally`). Todo se ejecutó en un sandbox propio con carga k6 constante.

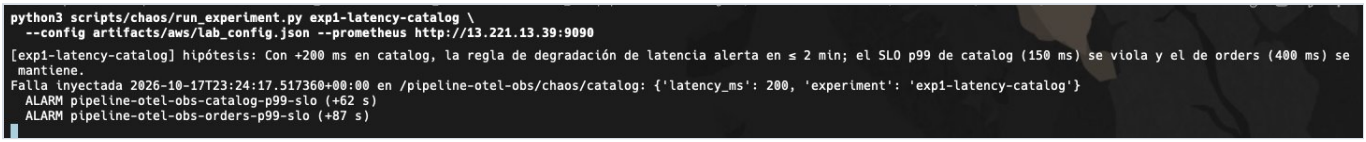


Figura 17. Ejecución del experimento 1: inyección de +200 ms en catalog y alarmas a +62 s y +87 s.

	Experimento 1: latencia en catalog (service-b)	**Experimento 2: errores en data-service**
Falla	`latency_ms = 200` durante 5 min	`error_rate = 0,10` (503 tras 500 ms) durante 5 min
Primera alarma (MTTD)	**catalog-p99-slo: 61,5 s**	**data-service-static-error-rate: 69,6 s**
Otras alarmas	orders-p99-slo: 87,0 s (efecto en cascada)	data-service-p99-slo: 123,3 s
Alarmas compuestas / anomalía	**No se dispararon**	**No se dispararon**
p99 base → durante la falla	48,5 ms → **249,5 ms** (SLO 150 ms)	61,3 ms → **725,4 ms** (SLO 300 ms)
Disponibilidad base → falla	100 % → 100 %	100 % → **89,86 %** (SLO 99 %)
Impacto aguas arriba (orders)	p99 487,7 ms (> SLO 400 ms), 0 errores	p99 1074 ms (> SLO 400 ms), 0 errores: orders degrada sin fallar
Recuperación tras revertir	2 s	2 s

Una segunda corrida del experimento 2 (3 min, grabada en el video) reprodujo el resultado: error-rate en 72,1 s, p99-slo en 125,8 s, disponibilidad 90,76 % y orders con p99 de 1262 ms.

Respuestas del Módulo D

Pregunta	**Experimento 1**	**Experimento 2**
¿Se degradó el SLO?	Sí, el de latencia: catalog p99 249,5 ms frente a 150 ms, y se propagó a orders (487,7 ms frente a 400 ms). La disponibilidad no se afectó.	Sí, ambos: disponibilidad 89,86 % frente a 99 % y p99 725 ms frente a 300 ms. orders conservó el 100 % de disponibilidad gracias a su manejo de errores (`data_record_failed`), pero violó su SLO de latencia.
¿Se consumió el error budget?	No: 0 % (no hubo errores, solo lentitud). El presupuesto de latencia sí se habría consumido si se midiera como SLO de latencia por ventana.	Sí: se consumió el **1014 %** del presupuesto de la ventana (≈10 veces lo permitido), equivalente al **0,12 % del presupuesto mensual** en 5 min. A ese ritmo sostenido, el presupuesto mensual se agotaría en unas 69 h (≈3 días).
¿La alerta fue accionable?	Parcialmente. La alarma identifica servicio, SLI y umbral, pero no llegó la alerta enriquecida porque la compuesta no se activó.	Parcialmente. El `trace_id` se obtuvo manualmente navegando del panel «Logs 5xx con trace_id» a Jaeger (demostrado en el video), no de forma automática.
MTTD ≤ 2 min	**Sí (62 s)**	**Sí (70 s)**

8. Reporte de madurez y roadmap (Módulo E)

Escala: 1 Inicial · 2 Gestionado · 3 Definido (estándares, correlación, SLO) · 4 Medido (detección validada, MTTD medido) · 5 Optimizado. Los dominios se derivan del enunciado y del blueprint.

#	Dominio	Nivel	Evidencia principal	Brecha para subir de nivel
1	Tres pilares y correlación	3	Trazas, métricas y logs con `trace_id` (Fig. 11–13)	Sin exemplars ni enlace traza → logs automático
2	Instrumentación OpenTelemetry	3	SDK 1.45.1, semconv HTTP y DB estables, spans de negocio	Sin muestreo configurable
3	DataOps de telemetría	2	Collector con límites y `servicegraph`	Sin tail sampling, control de cardinalidad ni costo por señal
4	Visualización y análisis	3	4 dashboards generados por código	Sin pruebas de dashboards
5	Alertamiento y AIOps	2	Alarmas estáticas validadas; anomalía y correlación desplegadas	Bandas sin datos: la correlación no se validó
6	Red y seguridad	2	Flow Logs, Security Hub, Inspector, alerta real de rechazos	Sin CloudTrail ni correlación hallazgo → servicio
7	SRE: SLO y error budget	3	SLO por servicio, burn rate, budget medido en caos	Sin política de error budget ni postmortems
8	Resiliencia y caos	3	2 experimentos con hipótesis, MTTD y reversión	Experimentos manuales, sin game days

Promedio: 2,6. Objetivo a 3 meses: **3,4**, subiendo los dominios 3, 5 y 6 a nivel 3 y los dominios 7 y 8 a nivel 4.

Mes	Iniciativa	Dominio	Indicador verificable
1	`default_value=0` en los filtros de error, entrenamiento ≥ 24 h y nueva validación de la regla correlacionada con exp. 2	5	correlated-errors en ALARM ≤ 120 s con `trace_id`
1	Tail sampling (100 % errores y lentos, 10 % resto) y límites de cardinalidad	3	−60 % de spans sin perder trazas de error
1	CloudTrail → CloudWatch con alertas de `AccessDenied` y `ConsoleLogin` fallido	6	Alerta ≤ 5 min ante 5 fallos
2	Política de error budget y alertas de burn rate multiventana (1 h/5 min, 6 h/30 min)	7	100 % de alertas de SLO ligadas a un budget
2	Supresión por dependencia en la Lambda (catalog lento → no alertar orders)	5	−30 % de notificaciones en fallas en cascada
3	Caos programado semanal y AWS FIS (red, parada de tareas) con aborto por SLO	8	4 ejecuciones/mes con MTTD ≤ 120 s
3	Despliegue de la referencia GCP y vista multinube en Grafana	1, 6	Mismos SLIs en ambas nubes

Conclusión. El sistema detecta fallas reales en menos de 2 minutos y permite llegar del síntoma (SLO) a la causa (traza y span de base de datos) en minutos. La principal lección es que una regla de AIOps solo es tan buena como los datos con los que se entrena: una métrica sin valor por defecto anuló la correlación. Por eso validarla con caos antes de confiar en ella resultó indispensable.